

Machine Data and Learning

Assignment-2

Bias-Variance Trade-off

Max Marks : 100

Deadline : 17th February, 2026

1 Introduction

1.1 Bias-Variance trade-off

When we discuss model prediction, it is important to understand the various prediction errors - bias and variance. There is a trade-off between a model's ability to minimise bias and variance. A proper understanding of these errors would help in distinguishing a layman and an expert in Machine Learning. Before using different classifiers, it is important to understand how to select a classifier to use.

Let us get started and understand some basic definitions that are relevant. For basic definitions, when $\hat{f}$ is applied to an unseen sample, refer [here](#).

- **Bias** is the difference between the average prediction of our model and the correct value that we are trying to predict. A model with high bias does not generalise the data well and oversimplifies the model. It always leads to a high error on training and test data. The formula for Bias for a given data point x is :

$$Bias = E_i[\hat{f}_i(x)] - f(x)$$

where $f(x)$ represents the true value, $\hat{f}_i(x)$ represents the value predicted by the i th model, $E_i[\cdot]$ is the expectation over all models.

- **Variance** is the variability of a model prediction for a given data point. Again, imagine you can repeat the entire model-building process multiple times. The variance is how much the predictions for a given point vary between different realisations of the model. The formula for Bias for a given data point x is :

$$Variance = E_i[(\hat{f}_i(x) - E_i[\hat{f}_i(x)])^2]$$

where $\hat{f}_i(x)$ represents the value predicted by the i th model, $E_i[\cdot]$ is the expectation over all models.

- **Noise** is any unwanted distortion in data. Noise is anything that is spurious and extraneous to the original data, that is not intended to be present in the first place but was introduced due to a faulty capturing process.
- **Irreducible error** is the error that cannot be reduced by creating good models. It is a measure of the amount of noise in the data. Here, it is important to understand that no matter how good we make our model, our data will have a certain amount of noise or irreducible error that cannot be removed.

$$E_i[(y - \hat{f}_i(x))^2] = Bias^2 + Variance + \sigma^2$$

$$\implies \sigma^2 = E_i[(y - \hat{f}_i(x))^2] - (Bias^2 + Variance)$$

where y represents the true value with some noise, $\hat{f}(x)$ represents the predicted value, $E_i[(f(x) - \hat{f}(x))^2]$ represents Mean Squared Error (MSE) at a given point x and σ^2 represents irreducible error.

Note : While reporting the values, you need to take the expectation of Bias, Variance and MSE over all data points. However these expected values may not hold the above relation.

$$E_x[MSE] \neq E_x[Bias]^2 + E_x[Variance] + E_x[\sigma^2]$$

but,

$$E_x[MSE] = E_x[Bias^2] + E_x[Variance] + E_x[\sigma^2]$$

If our model is too simple and has very few parameters then it may have high bias and low variance. On the other hand, if our model has a large number of parameters then it is going to have high variance and low bias. So we need to find the right (or good) balance without overfitting and underfitting the data.

1.2 Linear Regression

Linear Regression is a supervised machine learning algorithm where the predicted output is continuous and has a constant slope. It's used to predict values within a continuous range, (e.g. sales, price) rather than trying to classify them into categories (e.g. cat, dog). There are two main types:

- Simple regression
- Multi-variable regression

For a more detailed definition refer this [article](#).

For simple linear regression with only one feature the equation is :

$$y = wx + b$$

where,

- y = Predicted value/Target Value
- x = Input
- w = Gradient/slope/Weight
- b = Bias

Similarly for a Multi-variable regression model the equation is :

$$y = b + \sum_{i=1}^n w_i x_i$$

Once we have the prediction function we need to determine the value of weight(s) and bias. To see how to calculate the value of weight(s) and bias, refer this [article](#).

2 Tasks

Task 1: Gradient Descent

(a) Explain how gradient descent works to find the coefficients when there is one independent variable and one dependent variable.

(b) Extend your explanation to the case of multiple independent variables and one dependent variable. Describe how gradient descent is adapted to handle this scenario and find the coefficients in a multivariate regression model.

Task 2: Numerical on Bias and Variance

In this task you will be calculating the bias², variance and MSE for the given dataset and verifying the formula $MSE = Bias^2 + Variance$. (Assume irreducible error σ^2 to be 0)

Consider the dataset (x, y) given by:

$$x : [-2, -1, 0, 1, 2, 3]$$

$$y : [30, 14, 1, -3, -1, -8]$$

On training over different splits of data you get the following 3 models:

$$f_1(x) = 3x^2 - 9x - 1$$

$$f_2(x) = 2x^2 - 9x + 4$$

$$f_3(x) = -x^3 + 3x^2 - 6x + 2$$

Calculate the bias², variance and MSE and verify the above formula.

Provide detailed steps for the calculations and mention the formulas used.

Task 3: Calculating Bias and Variance

Over the past year, global supply chain disruptions have led to a shortage of DRAM (RAM) chips. As a result, laptop manufacturers have faced rising component costs, which are partially passed on to consumers.

You are a market analyst for one such laptop manufacturer and you are tasked with pricing the laptops in such a way that the company can maintain a good profit margin and not lose many sales.

The company wants you to model the relationship between **RAM prices** and **laptop prices** using historical data, so that the company can price their laptops appropriately.

In this task you need to train models on this data and evaluate their bias and variance so that you can choose a model which balances underfitting and overfitting and help you pick the right pricing.

3.1 How to Re-Sample data

You are given two datasets, i.e., a training set and a test set, consisting of pairs (x_i, y_i) , where x_i corresponds to the average RAM prices, and y_i corresponds to the average laptop prices. This data can be loaded into your Python program using the `pickle.load()` function. You then need to divide the training set into 15 equal parts randomly so that you get 15 different training datasets to train your model.

3.2 Task

After re-sampling the data, you have 16 different datasets (15 train sets and 1 test set). Train a linear classifier on each of the 15 train sets separately so that you have 15 different classifiers or models. Now you can calculate the bias and variance of the model using the test set. You need to repeat the above process for the following class of functions,

- $y = ax + b$
- $y = ax^2 + bx + c$
- $y = ax^3 + bx^2 + cx + d$

And so on, up till polynomials of degree 10. The only two functions that you are allowed to use are (from sklearn):

- `linear_model.LinearRegression().fit()`
- `preprocessing.PolynomialFeatures()`

These functions will help you find the appropriate coefficients with the default parameters. Tabulate the values of bias and variance and also write a detailed report explaining how bias and variance change as you vary your function classes.

Note: Whenever we are talking about the bias and variance of the model, it refers to the average bias and variance of the model over all the test points. For every degree, Bias and Variance are defined as the mean of biases and variances of all 15 models.

3.3 Task

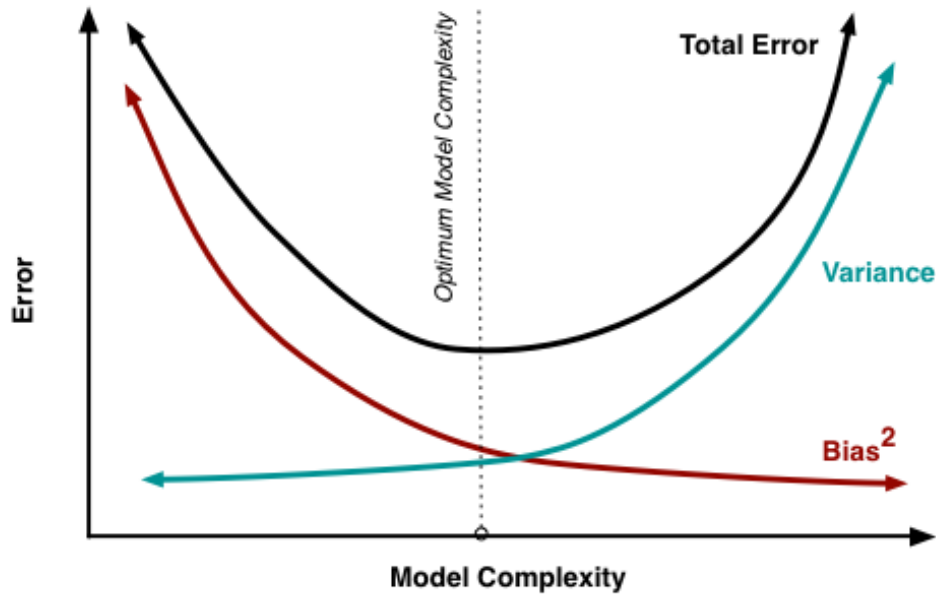
Repeat Tasks 3.1 and 3.2 by dividing the training dataset into 10 different training sets, and again into 20 different training sets. Which of these(10,15,20) do you think is better, and why?

Task 4: Calculating Irreducible Error

Tabulate the values of irreducible error for the models in Task 3 and also write a detailed report explaining why or why not the value of irreducible error changes as you vary your class function.

Task 5: Plotting Bias² - Variance Graph

Based on the variance, bias and total error calculated in earlier tasks, plot the Bias²-Variance tradeoff graph and write your observations in the report with respect to underfitting, overfitting and also comment on the type of data just by analysing the Bias²-Variance plot. The below figure shows the balance between model framework error and model complexity.



Plot variation of Bias², Variance and MSE against degree of polynomial in the same graph.

Task 6: Overfitting and Regularization

In this task, you will explore the effects of high-degree polynomial overfitting and implement a hybrid regularization strategy (an ensemble) to improve model generalization.

Important Note: Because polynomial features of high degree vary significantly in scale, you must normalize your features. Use `StandardScaler` after generating the polynomial features but before fitting any models.

- Polynomial Feature Expansion:** Transform the training data into polynomial features of **degree 15**. Apply `StandardScaler` to standardize these features. Fit a standard Linear Regression model (without regularization). Compute and report the Mean Squared Error (MSE) on the test set.
- Individual Regularized Models:** Train two separate regularized models on the standardized degree 15 features using the following fixed regularization strengths (λ):
 - Ridge Regression:** $\lambda = 0.1$ (set `alpha=0.1` in sklearn)
 - Lasso Regression:** $\lambda = 0.0001$ (set `alpha=0.0001` in sklearn)

Report the MSE for both models on the test set.

- Hybrid Model Optimization:** Implement a hybrid predictor $\hat{y}_{hybrid}$ defined by the linear combination of the predictions from the Ridge and Lasso models. Use a mixing parameter $\beta \in (0, 1)$ such that:

$$\hat{y}_{hybrid} = \beta \cdot \hat{y}_{Ridge} + (1 - \beta) \cdot \hat{y}_{Lasso}$$

Iterate through values of β in the interval $(0, 1)$ (exclusive of the trivial bounds 0 and 1) to find the **optimal non-trivial** β that minimizes the Mean Squared Error on the test set. Report the value of β_{opt} and the corresponding lowest MSE.

- (d) **Analysis:** Compare the MSE values obtained in parts (a), (b), and (c). Discuss why this weighted ensemble of L1 (Lasso) and L2 (Ridge) regularized predictions might outperform the individual models.

Constraints: You are only allowed to use the following functions from `sklearn`:

- `linear model.Ridge()`
- `linear model.Lasso()`
- `preprocessing.StandardScaler`
- `preprocessing.PolynomialFeatures`

3 General Instructions

- The data is given as pickle file. On loading the pickle file you will get a dictionary object which has training set with "train" key and test set with "test" key.
- Submit a zip file named `<rollnumber>_A2.zip` containing source code and the report:
 - `code.ipynb`
 - `report.pdf`
 - `readme.md` (if any assumptions)
- All coding has to be done in Python3 only, using Jupyter Notebook.
- Code is only required for Tasks 3, 4, 5 and 6.
- Report should include detailed answers for Tasks 1 and 2.
- Report should include all details needed for evaluation. Please include relevant graphs, tables, analysis, observations, and writeup as required for each of the tasks above.
- Get familiar with numpy, matplotlib, pickle, pandas dataframe, and sklearn.
- Plagiarism will be penalized heavily.
- Manual evaluations will be held regarding which further details will be announced later.

Marking Scheme

- Task 1: 5 marks
- Task 2: 5 marks
- Task 3: 25 marks
- Task 4: 10 marks
- Task 5: 15 marks
- Task 6: 10 marks
- Viva: 30 marks